

```
# Google Colab Bengali Character Fast 14M Classifier
# Architecture:
# Fast LeNet-style stem -> residual CNN stages -> lightweight
token/MLP mixer -> classifier.
# Designed for very fast training with about 1.4 crore parameters.
#
# Expected zip layout:
# Dataset.zip
#   train/
#     class_1/
#       image001.bmp
#     class_2/
#       image002.bmp
#   test/
#     class_1/
#       image003.bmp
#     class_2/
#       image004.bmp
#
# In Colab:
# 1. Runtime > Change runtime type > GPU
# 2. Paste/run this file in one notebook cell.
```

```
import os
import random
import shutil
import time
import zipfile
from pathlib import Path
from typing import Dict, List, Optional, Tuple

import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from google.colab import files
from PIL import Image
from sklearn.metrics import classification_report, confusion_matrix
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from torchvision.datasets import ImageFolder
```

```
SEED = 42
ZIP_LIMIT_MB = 500
DATA_ROOT = Path("/content/bengali_hybrid_dataset")
OUTPUT_DIR = Path("/content/bengali_hybrid_results")
```

```
# If upload button does not appear, upload zip from Colab's left Files
panel
```

```

# and set ZIP_PATH = "/content/your_file.zip".
ZIP_PATH = ""
AUTO_USE_NEWEST_CONTENT_ZIP = True

MODEL_IMAGE_SIZE = 64
BATCH_SIZE = 256
EPOCHS = 20
LEARNING_RATE = 2e-3
WEIGHT_DECAY = 1e-4
NUM_WORKERS = 4
MAX_TRAIN_IMAGES = 40000
MAX_TEST_IMAGES = 10000
USE_TORCH_COMPILE = True

def seed_everything(seed: int) -> None:
    random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True

def choose_zip_file() -> Path:
    if ZIP_PATH:
        zip_path = Path(ZIP_PATH)
        if not zip_path.exists():
            raise FileNotFoundError(f"ZIP_PATH does not exist:
{zip_path}")
        return zip_path

    existing_zips = sorted(Path("/content").glob("*.zip"), key=lambda
p: p.stat().st_mtime, reverse=True)
    if AUTO_USE_NEWEST_CONTENT_ZIP and existing_zips:
        zip_path = existing_zips[0]
        print(f"Using existing zip from /content: {zip_path}")
        return zip_path

    print("Upload your train/test zip file now.")
    print("If the upload button does not appear, upload the zip from
Colab's left Files panel and rerun.")
    uploaded = files.upload()
    if not uploaded:
        raise RuntimeError("No zip uploaded. Set ZIP_PATH or upload a
zip to /content and rerun.")
    zip_name = next(iter(uploaded.keys()))
    return Path("/content") / zip_name

def upload_and_extract_zip() -> Path:

```

```

zip_path = choose_zip_file()
if zip_path.suffix.lower() != ".zip":
    raise ValueError("Please upload a .zip file.")

size_mb = zip_path.stat().st_size / (1024 * 1024)
if size_mb > ZIP_LIMIT_MB:
    raise ValueError(f"Zip is {size_mb:.1f} MB. Limit is
{ZIP_LIMIT_MB} MB.")

if DATA_ROOT.exists():
    shutil.rmtree(DATA_ROOT)
DATA_ROOT.mkdir(parents=True, exist_ok=True)

with zipfile.ZipFile(zip_path) as archive:
    for member in archive.infolist():
        target = DATA_ROOT / member.filename
        if not
str(target.resolve()).startswith(str(DATA_ROOT.resolve())):
            raise ValueError(f"Unsafe path inside zip:
{member.filename}")
        archive.extractall(DATA_ROOT)

print(f"Extracted to: {DATA_ROOT}")
return DATA_ROOT

def find_split_dir(root: Path, split_name: str) -> Path:
    direct = root / split_name
    if direct.exists() and direct.is_dir():
        return direct

    candidates = [p for p in root.rglob("*") if p.is_dir() and
p.name.lower() == split_name]
    if not candidates:
        raise FileNotFoundError(f"Could not find a '{split_name}'
folder inside the zip.")
    return candidates[0]

def count_images(split_dir: Path) -> int:
    extensions = {".bmp", ".png", ".jpg", ".jpeg", ".tif", ".tiff"}
    return sum(1 for p in split_dir.rglob("*") if p.is_file() and
p.suffix.lower() in extensions)

class LimitedImageFolder(Dataset):
    def __init__(self, root: Path, transform, max_images:
Optional[int] = None):
        self.base = ImageFolder(str(root), transform=None)
        self.classes = self.base.classes

```

```

self.class_to_idx = self.base.class_to_idx
self.transform = transform

samples = list(self.base.samples)
if max_images and len(samples) > max_images:
    by_class: Dict[int, List[Tuple[str, int]]] = {}
    for sample in samples:
        by_class.setdefault(sample[1], []).append(sample)
    limited = []
    per_class = max(1, max_images // max(1, len(by_class)))
    rng = random.Random(SEED)
    for class_samples in by_class.values():
        rng.shuffle(class_samples)
        limited.extend(class_samples[:per_class])
    samples = limited[:max_images]

self.samples = samples
self.targets = [target for _, target in samples]

def __len__(self):
    return len(self.samples)

def __getitem__(self, index):
    path, target = self.samples[index]
    image = Image.open(path).convert("RGB")
    if self.transform:
        image = self.transform(image)
    return image, target

class LeNetStem(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=5, padding=2),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=5, padding=2),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        return self.features(x)

```

```

class ConvBNAct(nn.Module):
    def __init__(self, in_channels: int, out_channels: int, stride:
int = 1):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3,
stride=stride, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.SiLU(inplace=True),
        )

    def forward(self, x):
        return self.block(x)

class FastResidualBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int, stride:
int = 1):
        super().__init__()
        self.conv1 = ConvBNAct(in_channels, out_channels,
stride=stride)
        self.conv2 = nn.Sequential(
            nn.Conv2d(out_channels, out_channels, kernel_size=3,
padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
        )
        self.skip = (
            nn.Identity()
            if in_channels == out_channels and stride == 1
            else nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1,
stride=stride, bias=False),
                nn.BatchNorm2d(out_channels),
            )
        )
        self.act = nn.SiLU(inplace=True)

    def forward(self, x):
        return self.act(self.conv2(self.conv1(x)) + self.skip(x))

class Fast14MCharacterNet(nn.Module):
    def __init__(self, num_classes: int):
        super().__init__()
        self.stem = nn.Sequential(
            ConvBNAct(3, 64),
            FastResidualBlock(64, 64),
            FastResidualBlock(64, 64),
        )

```

```

        self.stage2 = nn.Sequential(
            FastResidualBlock(64, 128, stride=2),
            FastResidualBlock(128, 128),
        )
        self.stage3 = nn.Sequential(
            FastResidualBlock(128, 256, stride=2),
            FastResidualBlock(256, 256),
            FastResidualBlock(256, 256),
        )
        self.stage4 = nn.Sequential(
            FastResidualBlock(256, 512, stride=2),
            FastResidualBlock(512, 512),
        )
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.mixer = nn.Sequential(
            nn.Linear(512, 2048),
            nn.SiLU(inplace=True),
            nn.Dropout(0.20),
            nn.Linear(2048, 512),
            nn.SiLU(inplace=True),
        )
        self.classifier = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.stem(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.stage4(x)
        x = self.pool(x).flatten(1)
        x = self.mixer(x)
        return self.classifier(x)

def get_transforms() -> Tuple[transforms.Compose, transforms.Compose]:
    train_tfms = transforms.Compose(
        [
            transforms.Resize((MODEL_IMAGE_SIZE, MODEL_IMAGE_SIZE)),
            transforms.RandomAffine(degrees=8, translate=(0.05, 0.05),
scale=(0.95, 1.05)),
            transforms.ToTensor(),
            transforms.Normalize(mean=(0.485, 0.456, 0.406),
std=(0.229, 0.224, 0.225)),
        ]
    )
    test_tfms = transforms.Compose(
        [
            transforms.Resize((MODEL_IMAGE_SIZE, MODEL_IMAGE_SIZE)),
            transforms.ToTensor(),
            transforms.Normalize(mean=(0.485, 0.456, 0.406),
std=(0.229, 0.224, 0.225)),
        ]
    )

```

```

    ]
)
return train_tfms, test_tfms

def train_one_epoch(model, loader, criterion, optimizer, scheduler,
                    scaler, device) -> Tuple[float, float]:
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images = images.to(device, non_blocking=True,
memory_format=torch.channels_last)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)
        with torch.amp.autocast(device_type=device.type,
enabled=device.type == "cuda"):
            logits = model(images)
            loss = criterion(logits, labels)

            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            scheduler.step()

            total_loss += loss.item() * labels.size(0)
            correct += (logits.argmax(dim=1) == labels).sum().item()
            total += labels.size(0)

    return total_loss / max(1, total), correct / max(1, total)

@torch.no_grad()
def evaluate(model, loader, criterion, device) -> Tuple[float, float,
List[int], List[int]]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    y_true = []
    y_pred = []

    for images, labels in loader:
        images = images.to(device, non_blocking=True,
memory_format=torch.channels_last)
        labels = labels.to(device, non_blocking=True)
        logits = model(images)

```

```

        loss = criterion(logits, labels)
        preds = logits.argmax(dim=1)

        total_loss += loss.item() * labels.size(0)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
        y_true.extend(labels.cpu().tolist())
        y_pred.extend(preds.cpu().tolist())

    return total_loss / max(1, total), correct / max(1, total),
    y_true, y_pred

def fit_model(num_classes: int, train_loader, test_loader, device):
    model = Fast14MCharacterNet(num_classes).to(device,
memory_format=torch.channels_last)
    if USE_TORCH_COMPILE and hasattr(torch, "compile"):
        try:
            model = torch.compile(model)
            print("torch.compile enabled")
        except Exception as exc:
            print(f"torch.compile skipped: {exc}")

    trainable_params = [p for p in model.parameters() if
p.requires_grad]
    param_count = sum(p.numel() for p in trainable_params)
    print(f"Trainable parameters: {param_count:,} ({param_count /
10_000_000:.2f} crore)")

    try:
        optimizer = optim.AdamW(trainable_params, lr=LEARNING_RATE,
weight_decay=WEIGHT_DECAY, fused=device.type == "cuda")
    except TypeError:
        optimizer = optim.AdamW(trainable_params, lr=LEARNING_RATE,
weight_decay=WEIGHT_DECAY)
    scheduler = optim.lr_scheduler.OneCycleLR(
        optimizer,
        max_lr=LEARNING_RATE,
        epochs=EPOCHS,
        steps_per_epoch=len(train_loader),
        pct_start=0.15,
    )
    criterion = nn.CrossEntropyLoss(label_smoothing=0.05)
    scaler = torch.amp.GradScaler("cuda", enabled=device.type ==
"cuda")

    best_acc = 0.0
    best_true = []
    best_pred = []
    started = time.perf_counter()

```



```

    for epoch in range(1, EPOCHS + 1):
        train_loss, train_acc = train_one_epoch(model, train_loader,
        criterion, optimizer, scheduler, scaler, device)
        test_loss, test_acc, y_true, y_pred = evaluate(model,
        test_loader, criterion, device)
        if test_acc >= best_acc:
            best_acc = test_acc
            best_true = y_true
            best_pred = y_pred
        print(
            f"epoch {epoch:02d}/{EPOCHS} "
            f"train_loss={train_loss:.4f} train_acc={train_acc:.4f} "
            f"test_loss={test_loss:.4f} test_acc={test_acc:.4f}"
        )

    return {
        "model": "Fast14M_Residual_LeNetStyle_CNN",
        "parameters": param_count,
        "accuracy": best_acc,
        "train_seconds": round(time.perf_counter() - started, 2),
        "status": "ok",
        "y_true": best_true,
        "y_pred": best_pred,
    }

def main() -> None:
    root = upload_and_extract_zip()
    seed_everything(SEED)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    device = torch.device("cuda" if torch.cuda.is_available() else
    "cpu")
    print("Device:", device)
    if device.type != "cuda":
        print("Warning: GPU not detected. This hybrid is heavy and
        will be slow on CPU.")

    train_dir = find_split_dir(root, "train")
    test_dir = find_split_dir(root, "test")

    print("\nDataset folders")
    print("Train:", train_dir)
    print("Test: ", test_dir)
    print("Train image count:", count_images(train_dir))
    print("Test image count: ", count_images(test_dir))

    train_tfms, test_tfms = get_transforms()
    train_dataset = LimitedImageFolder(train_dir, train_tfms,

```

```

MAX_TRAIN_IMAGES)
    test_dataset = LimitedImageFolder(test_dir, test_tfms,
MAX_TEST_IMAGES)

    if train_dataset.classes != test_dataset.classes:
        print("Warning: train/test class folders differ.")
        print("Train classes:", train_dataset.classes)
        print("Test classes: ", test_dataset.classes)

    num_classes = len(train_dataset.classes)
    print("\nClasses:", num_classes)
    print("Train samples used:", len(train_dataset))
    print("Test samples used: ", len(test_dataset))
    print("\nFast path")
    print("64x64 image -> LeNet-style stem -> residual stages -> MLP
mixer -> classifier")

    train_loader = DataLoader(
        train_dataset,
        batch_size=BATCH_SIZE,
        shuffle=True,
        num_workers=NUM_WORKERS,
        pin_memory=torch.cuda.is_available(),
        persistent_workers=NUM_WORKERS > 0,
    )
    test_loader = DataLoader(
        test_dataset,
        batch_size=BATCH_SIZE,
        shuffle=False,
        num_workers=NUM_WORKERS,
        pin_memory=torch.cuda.is_available(),
        persistent_workers=NUM_WORKERS > 0,
    )

    payload = fit_model(num_classes, train_loader, test_loader,
device)
    leaderboard = pd.DataFrame([k: v for k, v in payload.items() if k
not in ["y_true", "y_pred"]])
    leaderboard.insert(0, "rank", [1])

    print("\nLeaderboard")
    display(leaderboard)

    leaderboard_path = OUTPUT_DIR /
"fast14m_character_leaderboard.csv"
    leaderboard.to_csv(leaderboard_path, index=False)
    print("Saved leaderboard:", leaderboard_path)

    class_names = train_dataset.classes
    y_true_names = [class_names[i] for i in payload["y_true"]]

```

```

y_pred_names = [class_names[i] for i in payload["y_pred"]]

print(f"\nBest model: {payload['model']}")
print(f"Best accuracy: {payload['accuracy']:.4f}")
print("\nClassification report")
print(classification_report(y_true_names, y_pred_names,
zero_division=0))

cm = pd.DataFrame(
    confusion_matrix(y_true_names, y_pred_names,
labels=class_names),
    index=class_names,
    columns=class_names,
)
print("\nConfusion matrix")
display(cm)

cm_path = OUTPUT_DIR / "best_model_confusion_matrix.csv"
cm.to_csv(cm_path)
print("Saved confusion matrix:", cm_path)
files.download(str(leaderboard_path))

if __name__ == "__main__":
    main()

```

Upload your train/test zip file now.

If the upload button does not appear, upload the zip from Colab's left Files panel and rerun.

<IPython.core.display.HTML object>

Saving Dataset.zip to Dataset.zip

Extracted to: /content/bengali_hybrid_dataset

Device: cuda

Dataset folders

Train: /content/bengali_hybrid_dataset/Dataset/train

Test: /content/bengali_hybrid_dataset/Dataset/test

Train image count: 12000

Test image count: 3000

Classes: 50

Train samples used: 12000

Test samples used: 3000

Fast path

64x64 image -> LeNet-style stem -> residual stages -> MLP mixer -> classifier

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/
dataloader.py:424: UserWarning: This DataLoader will create 4 worker
processes in total. Our suggested max number of worker in current
system is 2, which is smaller than what this DataLoader is going to
create. Please be aware that excessive worker creation might get
DataLoader running slow or even freeze, lower the worker number to
avoid potential slowness/freeze if necessary.
  self.check_worker_number_rationality()
```

torch.compile enabled

Trainable parameters: 14,474,866 (1.45 crore)

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/
dataloader.py:432: UserWarning: This DataLoader will create 4 worker
processes in total. Our suggested max number of worker in current
system is 2, which is smaller than what this DataLoader is going to
create. Please be aware that excessive worker creation might get
DataLoader running slow or even freeze, lower the worker number to
avoid potential slowness/freeze if necessary.
```

```
  self.check_worker_number_rationality()
```

```
W0530 21:23:24.014000 5443 torch/_inductor/utils.py:1731] [0/0] Not
enough SMs to use max_autotune_gemm mode
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py
:432: UserWarning: This DataLoader will create 4 worker processes in
total. Our suggested max number of worker in current system is 2,
which is smaller than what this DataLoader is going to create. Please
be aware that excessive worker creation might get DataLoader running
slow or even freeze, lower the worker number to avoid potential
slowness/freeze if necessary.
```

```
  self.check_worker_number_rationality()
```

```
epoch 01/20 train_loss=3.6631 train_acc=0.0634 test_loss=9.5764
test_acc=0.0333
```

```
epoch 02/20 train_loss=2.6803 train_acc=0.2555 test_loss=3.6602
test_acc=0.1553
```

```
epoch 03/20 train_loss=1.5614 train_acc=0.5970 test_loss=1.3520
test_acc=0.6830
```

```
epoch 04/20 train_loss=0.9604 train_acc=0.8219 test_loss=1.1232
test_acc=0.7540
```

```
epoch 05/20 train_loss=0.7261 train_acc=0.9003 test_loss=0.6978
test_acc=0.9180
```

```
epoch 06/20 train_loss=0.6344 train_acc=0.9314 test_loss=0.5868
test_acc=0.9503
```

```
epoch 07/20 train_loss=0.5645 train_acc=0.9543 test_loss=0.6384
test_acc=0.9193
```

```
epoch 08/20 train_loss=0.5256 train_acc=0.9672 test_loss=0.5614
test_acc=0.9527
```

```
epoch 09/20 train_loss=0.4947 train_acc=0.9744 test_loss=0.5242
test_acc=0.9673
```

```
epoch 10/20 train_loss=0.4721 train_acc=0.9825 test_loss=0.4757
```

```
test_acc=0.9770
epoch 11/20 train_loss=0.4551 train_acc=0.9868 test_loss=0.5011
test_acc=0.9703
epoch 12/20 train_loss=0.4471 train_acc=0.9888 test_loss=0.4832
test_acc=0.9767
epoch 13/20 train_loss=0.4370 train_acc=0.9916 test_loss=0.4557
test_acc=0.9843
epoch 14/20 train_loss=0.4262 train_acc=0.9935 test_loss=0.4523
test_acc=0.9837
epoch 15/20 train_loss=0.4209 train_acc=0.9950 test_loss=0.4503
test_acc=0.9820
epoch 16/20 train_loss=0.4170 train_acc=0.9961 test_loss=0.4425
test_acc=0.9857
epoch 17/20 train_loss=0.4139 train_acc=0.9967 test_loss=0.4424
test_acc=0.9853
epoch 18/20 train_loss=0.4123 train_acc=0.9968 test_loss=0.4420
test_acc=0.9857
epoch 19/20 train_loss=0.4121 train_acc=0.9970 test_loss=0.4403
test_acc=0.9853
epoch 20/20 train_loss=0.4105 train_acc=0.9976 test_loss=0.4402
test_acc=0.9857
```

Leaderboard

```
{"summary":{"\n  \"name\": \"    main()\",\n  \"rows\": 1,\n  \"fields\": [\n    {\n      \"column\": \"rank\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 1,\n        \"max\": 1,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"Fast14M_Residual_LeNetStyle_CNN\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"parameters\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 14474866,\n        \"max\": 14474866,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          14474866\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 0.9856666666666667,\n        \"max\": 0.9856666666666667,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          0.9856666666666667\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"train_seconds\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 505.56,\n        \"max\": 505.56,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          505.56\n        ],\n
```

```

{"semantic_type": "\n", "description": "\n", "column": "status", "properties": {"dtype": "string", "num_unique_values": 1, "samples": [{"ok"}]}, "semantic_type": "\n", "description": "\n", "column": "status", "properties": {"dtype": "string", "num_unique_values": 1, "samples": [{"ok"}]}, "type": "dataframe"}

```

Saved leaderboard:

/content/bengali_hybrid_results/fast14m_character_leaderboard.csv

Best model: Fast14M_Residual_LeNetStyle_CNN

Best accuracy: 0.9857

Classification report

	precision	recall	f1-score	support
172	0.98	0.98	0.98	60
173	0.98	1.00	0.99	60
174	1.00	0.95	0.97	60
175	0.98	1.00	0.99	60
176	0.95	0.98	0.97	60
177	0.98	1.00	0.99	60
178	1.00	0.98	0.99	60
179	1.00	0.98	0.99	60
180	0.98	1.00	0.99	60
181	1.00	1.00	1.00	60
182	1.00	1.00	1.00	60
183	0.98	0.98	0.98	60
184	0.98	0.98	0.98	60
185	0.98	1.00	0.99	60
186	0.97	0.97	0.97	60
187	1.00	0.98	0.99	60
188	0.98	1.00	0.99	60
189	0.98	1.00	0.99	60
190	0.98	1.00	0.99	60
191	0.98	1.00	0.99	60
192	0.98	0.98	0.98	60
193	0.98	1.00	0.99	60
194	1.00	0.98	0.99	60
195	1.00	0.98	0.99	60
196	1.00	0.98	0.99	60
197	0.92	0.97	0.94	60
198	0.95	0.97	0.96	60
199	0.95	0.98	0.97	60
200	1.00	1.00	1.00	60
201	0.98	0.97	0.97	60
202	0.96	0.90	0.93	60
203	1.00	0.98	0.99	60
204	0.98	0.98	0.98	60
205	1.00	0.98	0.99	60

206	1.00	1.00	1.00	60
207	0.97	1.00	0.98	60
208	0.97	0.93	0.95	60
209	1.00	0.98	0.99	60
210	1.00	1.00	1.00	60
211	0.98	1.00	0.99	60
212	0.98	1.00	0.99	60
213	1.00	0.95	0.97	60
214	0.98	1.00	0.99	60
215	1.00	1.00	1.00	60
216	1.00	0.98	0.99	60
217	1.00	1.00	1.00	60
218	1.00	1.00	1.00	60
219	0.98	1.00	0.99	60
220	1.00	1.00	1.00	60
221	0.98	0.97	0.97	60
accuracy			0.99	3000
macro avg	0.99	0.99	0.99	3000
weighted avg	0.99	0.99	0.99	3000

Confusion matrix

```
{"type": "dataframe"}
```

Saved confusion matrix:

/content/bengali_hybrid_results/best_model_confusion_matrix.csv

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>